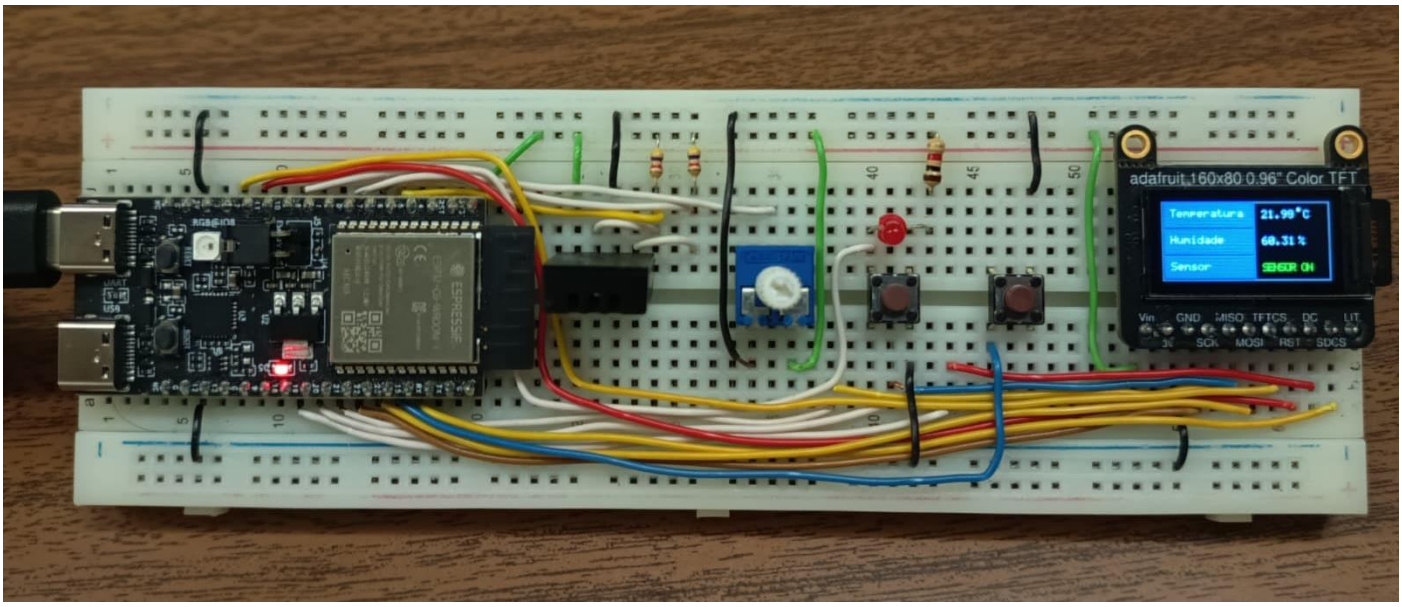
universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Dispositivos Conectados

Projeto Final (2025/2026)

Autores: Gabriel Lameira Marta (120155); Paulo Lacerda (120202)



1 Índice

1 Índice ---------- 2
2 Introdução ---------- 3
3 Componentes ---------- 4
3.1 ESP32-C6 DevKitC-1 ---------- 4
3.2 Sensor DHT20 ---------- 4
3.3 Display TFT ---------- 5
3.4 Botão e potenciómetro analógicos ---------- 5
4 Projeto ---------- 6
4.1 Requisitos do Projeto ---------- 6
5 Manual de Utilização ---------- 7
6 Arquitetura ---------- 8
6.1 Arquitetura Física / Hardware ---------- 8
6.2 Estrutura geral do projeto ---------- 9
6.2.1 Tarefas FreeRTOS (responsabilidades e prioridades) ---------- 15
6.2.2 Comunicação entre tarefas ---------- 16
6.2.3 Estruturas partilhadas na memória ---------- 16
6.2.4 Fluxo global do sistema ---------- 18
7 Conclusão ---------- 19

2 Introdução

Os sistemas *embedded* estão presentes em praticamente todos os domínios tecnológicos, como por exemplo, na automação industrial, nas telecomunicações, nos dispositivos médicos, nos meios de transporte, na produção de energia, nos eletrodomésticos e por fim, no domínio em que nos focamos neste projeto, a Internet das coisas (IoT). Esse conceito, que se tem popularizado nos últimos anos, baseia-se em ligar dispositivos físicos, com utilidade no nosso quotidiano (como eletrodomésticos, sensores, meios de transporte, entre outros) à Internet. Esses dispositivos devem ser autónomos e capazes de receber e enviar dados através da rede, reduzindo a intervenção humana ao mínimo. Essa autonomia e capacidade de comunicação com a rede permitem que os dispositivos operem de forma mais inteligente e que seja feita uma automatização de tarefas. A importância destes dispositivos provém da forma de operar mais inteligente acima referida, aliada a um consumo energético mais eficiente e aumento da fiabilidade a executar tarefas específicas para quais o sistema foi desenhado. Todas estas qualidades dos dispositivos assentam numa característica muito importante dos sistemas *embedded*: o facto de estes integrarem o hardware e o software necessário para executar as tarefas pretendidas no mesmo sistema dedicado.

Num contexto tecnológico em que a conectividade, a troca de dados e a autonomia assumem um papel fulcral, os sistemas *embedded* surgem como uma alavanca para a inovação digital. São estes mesmos sistemas que permitem transformar sensores e atuadores em sistemas inteligentes, capazes de comunicar, aprender e responder a eventos em tempo real.

3 Componentes

3.1 ESP32-C6 DevKitC-1

É uma placa de desenvolvimento (Fig. 3.1.1) baseada no módulo ESP32-C6-WROOM-1(U), produzido pela Espressif, que integra o chip ESP32-C6, o primeiro da marca baseado na arquitetura RISC-V. O módulo inclui uma flash SPI de 8 MB e um processador single-core de 32 bits a 160 MHz, oferecendo suporte para Wi-Fi 6, uma funcionalidade central para o projeto. Este módulo é amplamente utilizado no desenvolvimento de soluções IoT devido às suas capacidades de conetividade.

No projeto, a ligação aos periféricos é feita através da montagem da placa numa placa de ensaio. A placa integra ainda um LED de power ON, botões de Boot e Reset e um LED RGB endereçável. Dispõe também de uma porta USB Type-C, compatível com USB 2.0, utilizada para alimentação, gravação de firmware e comunicação com o PC/Host.

Figura 3.1.1 - ESP32-C6 DevKitC-1

3.2 Sensor DHT20

O sensor DHT20 (Fig.3.2.1), desenvolvido pela ASAIR, mede a humidade relativa e a temperatura do ar. O sensor já fornece os dados lidos com calibração e com a conversão de analógico para digital já feita, sendo a comunicação entre o sensor e o microcontrolador efetuada através do protocolo I^2C.

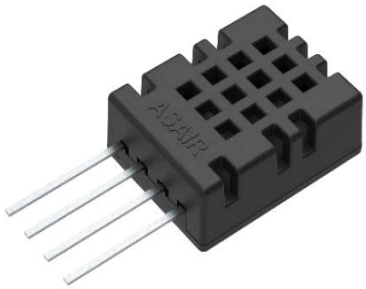


Figura 3.2.1 - Sensor DHT20

3.3 Display TFT

Os *displays* TFT são muito úteis em projetos de IoT devido à sua oferta de cores vivas, boa resolução e taxas de atualização elevadas. No nosso projeto, utilizamos o *display* Adafruit Mini TFT (Fig. 3.3.1) de 0,96 polegadas. A sua resolução é de 160x80 pixéis (cada pixel pode exibir cores RGB). Este *display* comunica através de SPI com apenas 4 fios e mantém numa memória interna, a imagem a exibir, o que faz com que seja compatível com vários microcontroladores, até com os mais simples. A placa inclui também um espaço para inserir um microSD, onde o microcontrolador pode escrever ou ler dados.

Figura 3.3.1, 3.3.2, 3.3.3 - Display Adafruit Mini TFT

3.4 Botão e potenciómetro analógicos

Dois componentes muito simples: o botão emite o sinal *high* enquanto é premido; o potenciómetro ajusta o sinal emitido consoante a rotação que lhe é aplicada (o sinal emitido é enquadrado dentro de uma escala ajustada à utilidade pretendida).

Figura 3.4.1 - Botão analógico

Figura 3.4.2 - Potenciómetro analógico

4 Projeto

O projeto consiste na conceção e implementação de um sistema *embedded* autónomo baseado no microcontrolador ESP32-C6 DevKitC-1. O dispositivo deve recolher dados de temperatura e humidade do sensor DHT20, exibi-los num *display* TFT e usar um LED RGB para indicar o estado do sistema. A recolha de dados é interrompida enquanto o botão de standby é premido. Os dados recolhidos podem ser registados no microSD presente no *display* TFT, dependendo da configuração definida pelo utilizador, para análise posterior e publicados via Wi-Fi num *broker* MQTT, o que permitirá a visualização dos mesmos em tempo real. Além de guardar os dados recolhidos, o microSD também manterá os *logs* das operações realizadas pelo sistema, estabelecendo assim um mecanismo de auditoria. O ajuste do brilho do ecrã é feito através de um potenciómetro analógico, de forma a garantir uma interface ajustável ao utilizador e uma maior eficiência energética.

As configurações do processo são armazenadas em ficheiros JSON presentes no microSD, o que torna possível ajustar o comportamento do sistema sem ser necessário recompilar todo o *firmware*, aumentando a flexibilidade e facilitando a manutenção.

Para a criação de uma interface de interação humano-computador, é feito o arranque de dois servidores. HTTP: um na rede do microcontrolador, para a configuração da rede Wi-Fi a que o mesmo se vai ligar; o outro, HTTPS na rede a que o microcontrolador se ligar, que servirá como *dashboard*.

4.1 Requisitos do Projeto

Requisitos Funcionais
Leitura da temperatura e humidade com período configurável
Visualização de dados no *display* TFT
Controlo do brilho do ecrã do display através do potenciómetro
Indicação do estado no LED RGB
Registo dos *logs* de eventos no microSD
Publicação das leituras e logs num broker MQTT através de Wi-Fi
Armazenar, no microSD, as configurações do sistema em ficheiros JSON
Implementação de estratégias de redução do consumo energético
Possibilidade de interromper as leituras pressionando o botão
Navegação entre ecrãs do display com botão
Configuração das credenciais da rede Wi-Fi através de um servidor HTTP
Visualização de um dashboard num servidor HTTPS

Requisitos Não Funcionais
O código está documentado e organizado em módulos
Tem um tempo de resposta adequado à funcionalidade
Tem uma interface simples e intuitiva
Não precisa de recompilar para alterar configurações
Adota práticas de segurança robustas

5 Manual de Utilização

Para o utilizador poder ter um guia de como proceder ao utilizar o nosso sistema, segue-se aqui uma lista de ações que pode realizar:

- Primeiro, o sistema deve ser ligado à corrente via USB-C. Caso tenham sido feitas alterações ao firmware, pode também ser utilizado o botão de reset para reiniciar o dispositivo.
- Após o arranque, num dispositivo ligado à rede Soft AP criada pelo sistema, deve ser aberto um browser e aceder o endereço **http://192.168.4.1**. Este servidor HTTP permite introduzir as credenciais da rede Wi-Fi e reiniciar o dispositivo.
- Se as credenciais forem válidas, o sistema inicia o servidor HTTPS, cujo URL é indicado na página HTTP. Este servidor funciona como dashboard e pode ser acedido através de um browser a partir de um dispositivo ligado à mesma rede, previamente configurada pelo servidor HTTP.
- Nesse servidor pode alterar o período de leitura do sensor, assim como visualizar várias estatísticas relacionadas com o sistema, como as leituras efetuadas, algumas configurações do sistema e o estado de alguns dos seus componentes.
- O utilizador pode ainda pressionar o botão físico para interromper ou retomar as leituras do sensor. O estado do sistema é indicado por um LED RGB: verde em funcionamento normal, amarelo durante a transmissão para o broker MQTT e vermelho quando ocorre algum erro.
- As publicações enviadas para o broker MQTT podem ser consultadas por qualquer cliente ligado à mesma rede Wi-Fi da placa de desenvolvimento, através de uma aplicação específica (MQTTX) ou diretamente pelo terminal, utilizando o endereço IP e a porta do broker. Estão disponíveis tanto as leituras do sensor como os logs de eventos, publicados respetivamente nos tópicos **"data"** e **"eventos"**.
- Além disso, é possível visualizar no display TFT as leituras efetuadas em tempo real, os logs de eventos e um gráfico com as 30 últimas leituras. O brilho do ecrã é controlado através do potenciómetro e a navegação entre ecrãs é feita com um botão, que também permite acordar o display após períodos de inatividade. Quando não é detetada qualquer interação, nem do potenciómetro nem do botão, durante 30 segundos, o brilho do ecrã é automaticamente reduzido. Se a inatividade se prolongar por mais 15 segundos, o display é desligado. Qualquer interação reinicia a contagem de inatividade e volta a ligar o ecrã.

6 Arquitetura

A arquitetura de software (Fig 6.1) foi feita de forma modular, o que torna mais fácil manter, testar e adicionar novas funcionalidades. O sistema está organizado em pastas, cada uma com a sua função bem definida. No hardware, as ligações foram feitas de forma direta entre os headers da placa e os pinos dos componentes, garantindo que tudo funciona como planeado. A disposição dos elementos foi pensada para ser o mais simples e limpo possível, facilitando a compreensão e evitando confusão nas ligações.

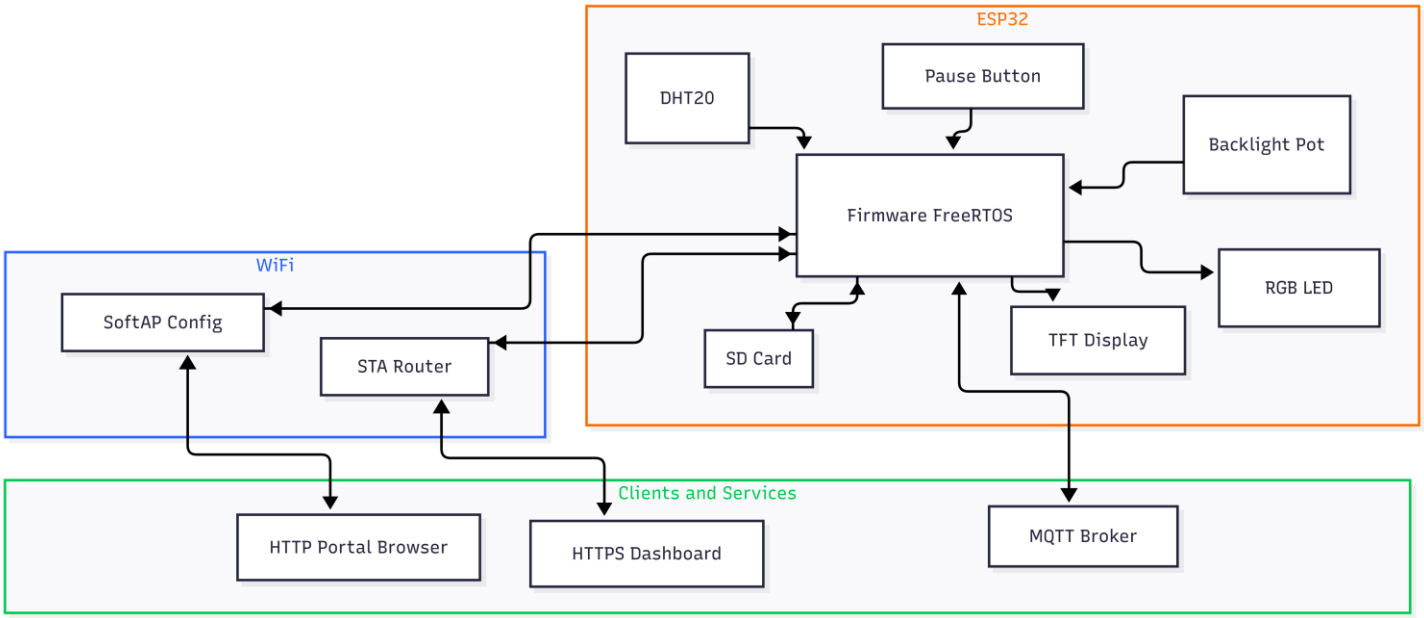


Figura 6.1 – Arquitetura global do sistema baseado no ESP32, incluindo periféricos, conectividade Wi-Fi e serviços externos.

6.1 Arquitetura Física / Hardware

A arquitetura física (Fig 6.1.1) do sistema é constituída pelo microcontrolador **ESP32-C6** e um conjunto de periféricos que garantem a recolha de dados ambientais, a interação com o utilizador e a comunicação com o exterior. Para além do microcontrolador, o sistema integra um sensor DHT20 para medir temperatura e humidade, e um display Adafruit TFT 160×80, utilizado como interface visual. Foram ainda incorporados dois botões de pressão, um potenciómetro para ajuste do brilho do ecrã, um LED para indicação de estado e a respetiva resistência de limitação de corrente. Estes elementos, em conjunto, constituem a base funcional do hardware do projeto.

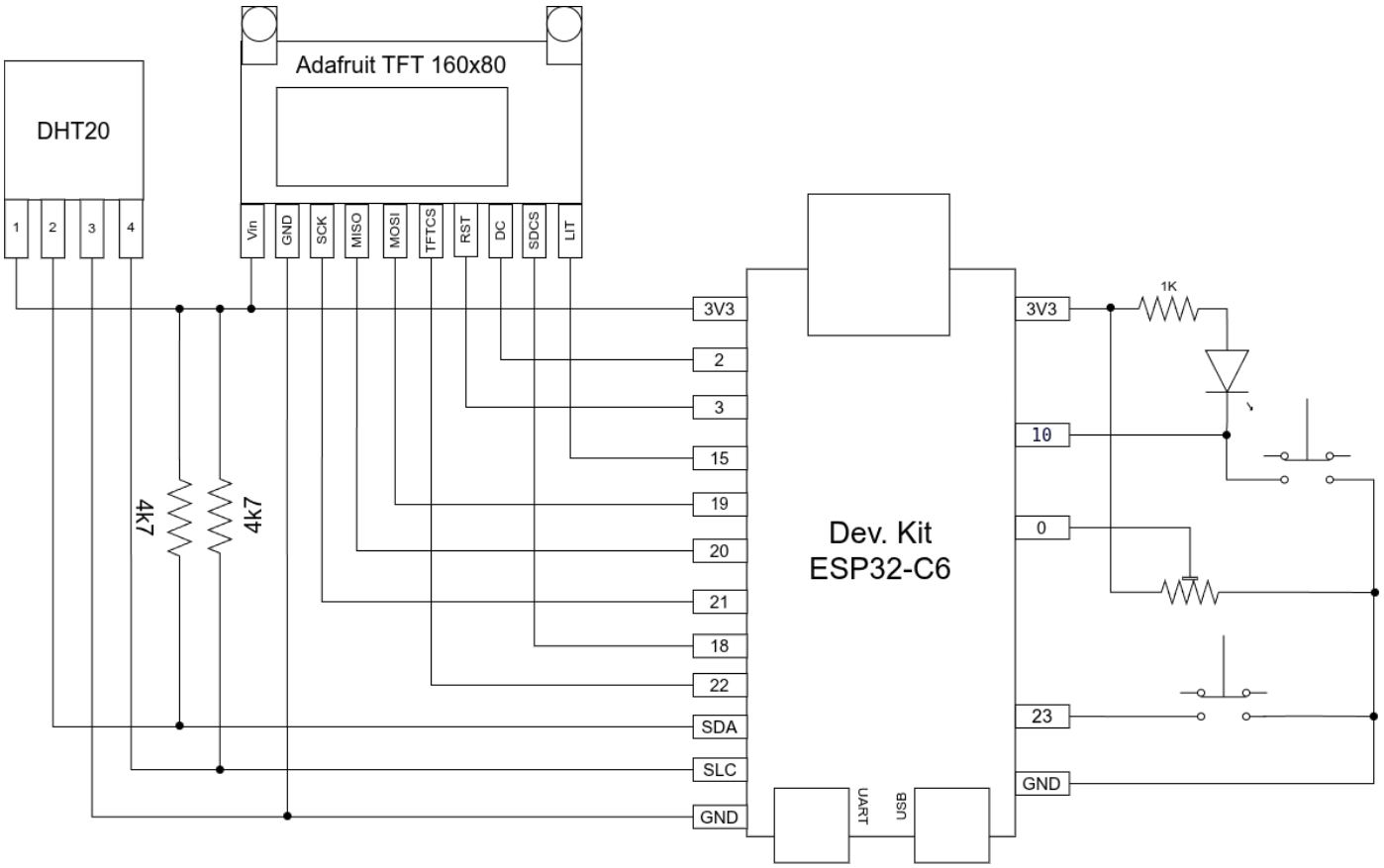


Figura 6.1.1 – Diagrama elétrico do sistema com ESP32-C6, DHT20, TFT ST7735, potenciómetro, LED RGB e botão.

6.2 Estrutura geral do projeto

O código encontra-se estruturado em pastas específicas, cada uma responsável por um conjunto de funcionalidades específicas:

main/core/

- **main.c**: ponto de entrada do programa; inicialização do hardware, leitura das configurações e criação das tarefas principais. Após a inicialização do estado global (system_status_init) e da configuração ativa (config_runtime_init), o sistema ativa o módulo de logging. O período de leitura do sensor é carregado e validado. Apenas se for válido são criadas as restantes tarefas FreeRTOS, nomeadamente:
- **task_brightness** — controlo do brilho via potenciómetro
- **task_display** — atualização do ecrã TFT
- **task_button_screen** — alternância entre ecrãs
- **task_sensor** — leituras periódicas do DHT20
- **task_mqtt** — publicação MQTT

O sistema tenta montar o cartão SD e, caso existam configurações de Wi-Fi válidas, estabelece ligação ao router; caso contrário, inicia o modo SoftAP para permitir configuração via portal. São depois

arrancados os servidores HTTP e HTTPS que disponibilizam o portal de configuração e o dashboard, respetivamente.

- **logging.c**: Este módulo suporta três níveis de log (INFO, WARN e ERROR) e garante que cada linha escrita no ficheiro **system.log** é acompanhada de um **HMAC-SHA256**, assegurando a integridade das entradas. O módulo cria um mutex interno para proteger o acesso concorrente ao ficheiro e utiliza buffers dinâmicos para formatar mensagens de qualquer tamanho.

```
17:57:24 | WARN | task_mqtt | MQTT desligado. | HMAC=8967bef8360cd1f620ebe2d34c1e7bd1949c0807ba103fb3fb5cb22c33c259d9

         |        |      |              |                                                |

Timestamp |  level  |  tag   |  message   |                                              HMAC
```

Quando uma mensagem é registada, esta é enviada simultaneamente para:

- o terminal (via esp_log),
- o display (através da função display_add_log),
- e o ficheiro no cartão SD.

- **system_status.c**: implementa o sistema central de monitorização do estado interno do dispositivo. Este módulo mantém uma estrutura chamada *system_status_snapshot_t*, onde são guardadas informações atualizadas sobre vários componentes do sistema, incluindo Wi-Fi, MQTT, cartão SD e dados gerais do dispositivo. Todas as operações são protegidas por um mutex, garantindo que múltiplas tarefas podem ler e atualizar o estado sem causar inconsistências.
- **led.c**: controlo do LED, permitindo indicação visual dos diferentes estados operacionais. A função central é led_set_state_with_reason(), que traduz o estado pretendido nas componentes RGB e chama rgb_led_show() para enviar os valores para o LED via RMT (Remote Control Peripheral).
  O módulo suporta quatro estados principais, representados pelo tipo *led_state_t*:
  - **LED_STATE_OFF** — LED desligado
  - **LED_STATE_NORMAL** — indica funcionamento regular (cor verde)
  - **LED_STATE_ERROR** — indica erro no sistema (cor vermelha)
  - **LED_STATE_TRANSMISSION** — indica atividade de envio MQTT (cor amarelo, mistura de vermelho+verde)

```
static led_state_t current_state = LED_STATE_OFF;
```

Figura 6.2.1 – Variável que mantém o estado atual do LED, iniciada como desligado.

main/sensor/

- **sensor_dht20.c**: driver para comunicação com o sensor DHT20 através de I²C.
- **task_sensor.c**: tarefa encarregue de efetuar leituras periódicas do sensor.

A tarefa do sensor executa sempre o mesmo ciclo de funcionamento (Fig 6.2.2):

1. **Verificação do cartão SD**
   Confirma se o SD está disponível. Se não estiver, a tarefa entra em espera e retoma apenas quando o cartão volta a ser detetado.
2. **Tratamento de notificações**
   Reage a pedidos de pausa e retoma (via botão) e a alterações do período de amostragem vindas da configuração.
3. **Validação do período de leitura**
   Garante que o intervalo configurado é válido antes de continuar (1 - 3600 s).
4. **Leitura do DHT20**
   a. Se a leitura tiver sucesso: guarda os valores, escreve no SD (se ativado), atualiza o display e notifica o módulo MQTT.
   b. Se falhar: regista o erro, marca os dados como inválidos e muda o LED para estado de erro.
5. **Espera até ao próximo ciclo**
   Aguarda o período configurado, a menos que uma nova notificação obrigue a ajustar o comportamento.

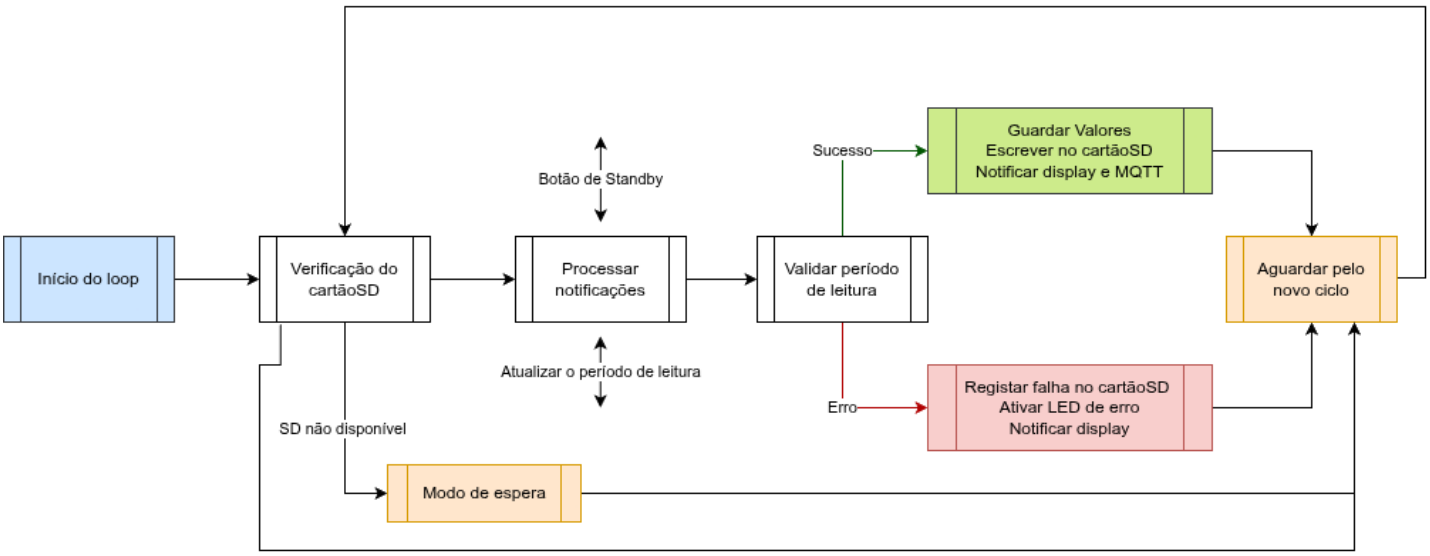


Figura 6.2.2 – Fluxo de funcionamento da tarefa de leitura do sensor (task_sensor).

- **sensor_data_store.c**: o módulo é responsável por armazenar e disponibilizar, de forma segura, as últimas leituras do sensor DHT20. Como estas leituras são utilizadas por várias tarefas (display, MQTT, logging, etc.), o módulo implementa um mecanismo de proteção através de um mutex, garantindo acesso concorrente seguro à informação.

  A estrutura interna guarda três elementos principais:

  - Última temperatura e humidade lidas (*s_latest*)
  - Indicação se a leitura atual é válida (*s_has_valid_data)*
  - Timestamp da última leitura (*s_last_timestamp_ms*)

Main/display

- **display.c**: responsável por gerir toda a interface gráfica apresentada no ecrã TFT. A função é desenhar a tabela principal com temperatura, humidade e estado do sensor, mostrar logs recentes e renderizar o gráfico com a evolução das leituras. Mantém buffers internos para armazenar logs e valores do gráfico, guardando assim os valores mesmo quando o ecrã está ativo. Inclui ainda funções para mensagens especiais, como aviso de SD removido ou ecrã de boas-vindas.
- **task_display.c**: tarefa responsável pela atualização visual do ecrã com base nas informações fornecidas pelo sistema.

```
typedef enum {
    SCREEN_MAIN = 0,          // Tabela principal
    SCREEN_LOGS,              // Logs recentes
    SCREEN_GRAPH_TEMP_HUM,    // Gráfico temp/hum
    SCREEN_MAX                // Total de ecrãs
} display_screen_t;
```

Figura 6.2.3 – Estrutura dos módulos responsáveis pela interface gráfica.

- **task_button_screen.c**: A tarefa lê periodicamente o GPIO do botão, aplicando debouncing para garantir deteções fiáveis. Sempre que o botão é premido, avança para o ecrã seguinte, usando aritmética modular para fazer a navegação circular entre os modos disponíveis. Após mudar o ecrã, envia uma notificação para a task_display, que trata de redesenhar o conteúdo no TFT (isto também faz com que o display fique acordado).

main/wifi/

- **task_mqtt.c**: implementação da ligação ao *broker* MQTT, publicação das mensagens e gestão de reconexões. Coloca o LED em estado de transmissão aquando da publicação de mensagens. Esta *task* é iniciada pela sua função de *start* que prepara parâmetros e lê o certificado do cartão SD. Este módulo assina as mensagens usando o UID do dispositivo e HMAC. Por fim, também faz a publicação de eventos do log (no tópico events) e das leituras do sensor (tópico data), com QoS igual a 0.
- **mqtt_utils.c**: utilitários para criação de tópicos, construção de payloads e conversão de dados. É responsável por ler o certificado do cartão SD para a memória heap.

main/config/

- **config_manager.c**: O módulo centraliza toda a lógica de configuração do dispositivo (Fig 6.2.4). Garante o acesso seguro ao cartão microSD, lê e valida ficheiros JSON de configuração, aplica defaults caso estejam corrompidos e gere credenciais Wi-Fi/MQTT, incluindo cifragem AES. Controla ainda o arranque de Wi-Fi em modo STA e SoftAP, mantendo uma lista de clientes ligados ao AP. Providencia funções de utilidade para obter e atualizar o período de leitura do sensor, UID do dispositivo e sincronização de hora via SNTP. É, na prática, o módulo que coordena toda a configuração persistente do sistema.

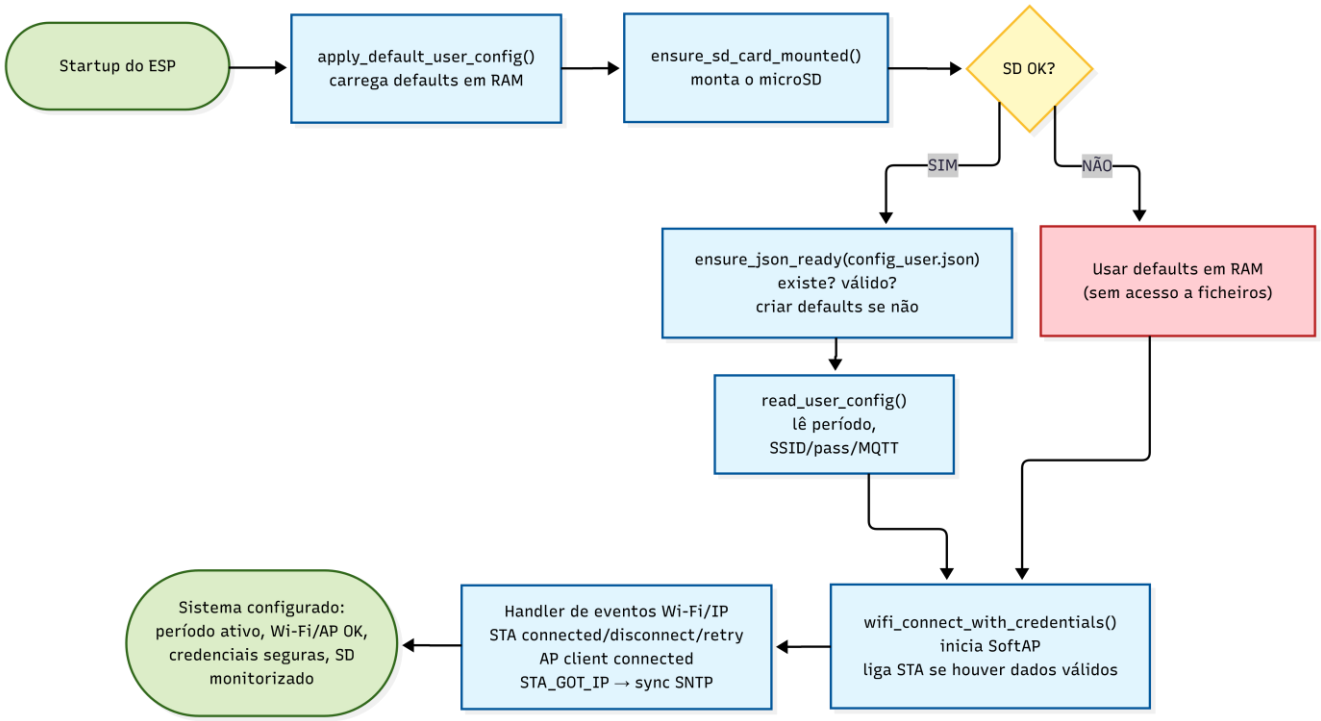


Figura 6.2.4 — Processo de arranque do ESP32 e lógica de validação/carregamento da configuração a partir do microSD.

- **config_runtime.c**: este módulo mantém configurações voláteis que podem ser alteradas em tempo de execução sem recorrer ao cartão SD. A variável global g_enable_sd_logging controla se o sistema deve guardar ou não as leituras no ficheiro data.txt, e o acesso a esta flag é protegido por um mutex para evitar conflitos entre tarefas. A função config_runtime_get_sd_logging_enabled() permite consultar o estado atual desta opção, enquanto config_runtime_set_sd_logging_enabled() atualiza o seu valor de forma segura.
- **task_config.c**: esta task trata, de forma exclusiva e segura, todos os pedidos de inicialização e recarregamento da configuração guardada no microSD. Outras tarefas enviam comandos para esta task, que chama o config_manager e devolve o resultado ao remetente. Assim evita-se acesso concorrente ao SD, garantindo assim uma utilização eficiente de energia e assegurando que a configuração é atualizada de forma consistente.

main/server/

- **config_portal_server.c:** fornece um portal HTTP (Fig 6.2.5) no SoftAP da placa de desenvolvimento para introdução das credenciais da rede Wi-Fi à qual nos queremos ligar. É responsável por guardar no cartão SD o SSID e a password dessa mesma rede Wi-Fi e de sinalizar o config_manager, para que este tente iniciar uma conexão. Também é possível efetuar um reset da placa a partir deste servidor.

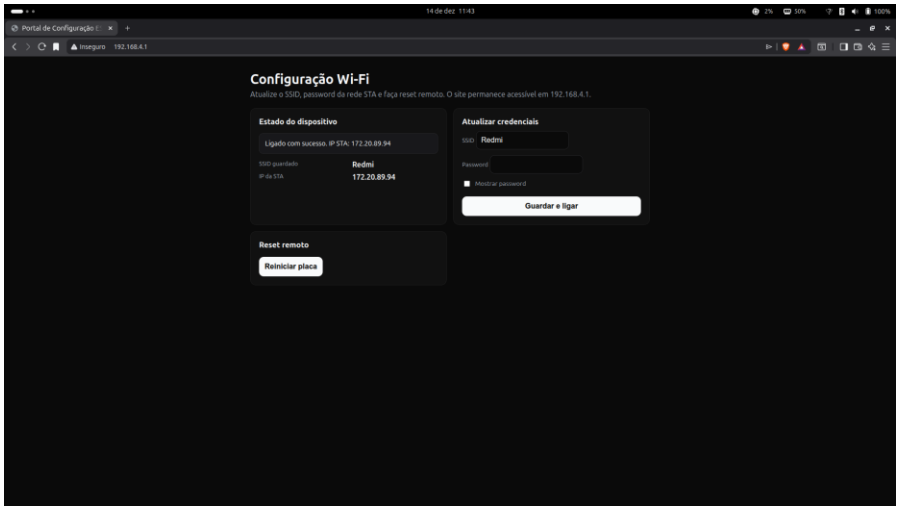


Figura 6.2.5 — Servidor HTTP

- **https_server_app.c:** implementa o servidor HTTPS (Fig 6.2.6) e as API's responsáveis pelo dashboard. Neste módulo está contida a API responsável por exibir as leituras do sensor, atualizando os gráficos em tempo real. Exibe também o estado global do sistema (Wi-Fi, broker, SD, clientes do SoftAP). Faz a resolução do IP/MAC dos clientes para poder escrever os logs de eventos. Permite alterar o período de leitura do sensor (clientes ligados ao SoftAP não têm permissão para efetuar alterações de configuração).

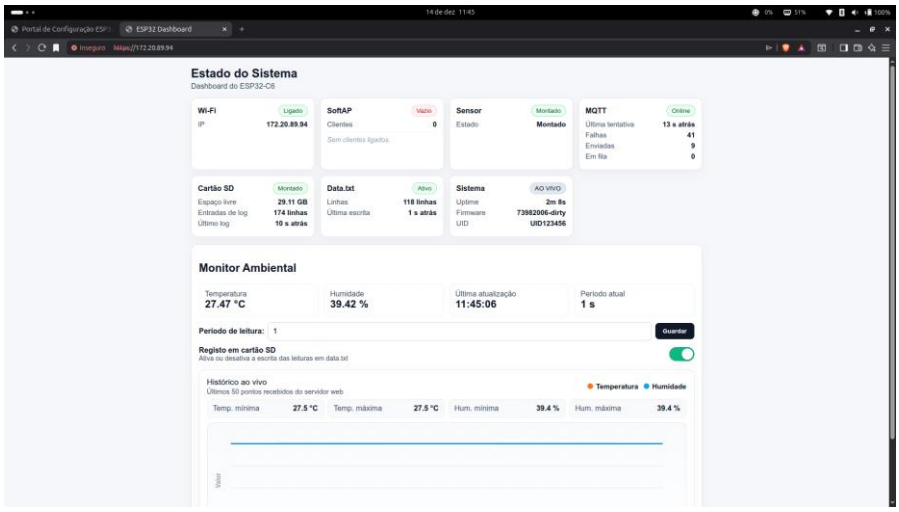


Figura 6.2.6 — Servidor HTTPS

Main/security/

- **security_hmac.c**: calcula o HMAC-SHA256 com a chave guardada no eFuse, necessário para a assinatura de mensagens do broker MQTT
- **security_aes.c**: implementa a cifra e decifra AES-256-CBC com a chave guardada no eFuse, necessária para cifrar as credenciais Wi-Fi antes de as escrever no cartão SD.

main/brightness/

- **task_brightness.c**: é a *task* responsável por ajustar o brilho do *display* TFT, com base no potenciómetro analógico. Faz parte do módulo da interface gráfica responsável pela *backlight* do display. O *duty-cycle* do PWM do microcontrolador é que controla o brilho dos LEDs do *backlight*. Esta *task* faz então a conversão da tensão fornecida pelo potenciómetro em valores de *duty-cycle* válidos. A *task* em si é definida então na função *task_brightness* que lê os valores do potenciómetro e ajusta o PWM se o valor mudar. Dentro deste módulo, também são aplicadas estratégias de redução do consumo energético: a task verifica se houve atividade externa no último ciclo, se não houver durante 30s, reduz o brilho do backlight; se passarem mais 15s sem qualquer atividade, desliga o backlight. Esta *task* corre em *background* e é iniciada pela função *task_brightness_start*, que inicializa o PWM e cria a *task* de monitorização do brilho. Esta função é invocada pelo *main* mal o *display* esteja disponível.

main/button/

- **task_button.c:** task responsável pela monitorização do botão físico e pela sinalização de eventos à task de controlo do sensor (*task_sensor*). Este módulo integra o mecanismo de interação com o utilizador. A task realiza polling periódico do GPIO do botão, aplicando debouncing por software para garantir deteções fiáveis e minimizar o impacto de ruído elétrico. A função *task_button_start* é responsável pela criação da task, sendo invocada no *app_main* após a configuração do respetivo GPIO.

main/include/

- **board_pins.h**: este ficheiro faz o mapeamento entre constantes, com nomes relativos à função física a que correspondem, e pinos GPIO da placa de desenvolvimento.

components/Driver-ST7735-ESP32-C6-main/

- Driver fornecido pelo docente e adaptado para o ST7735, usado como base para todo o módulo de display.

6.2.1 Tarefas FreeRTOS (responsabilidades e prioridades)

Tarefa	Função	Prioridade
task_sensor	Lê periodicamente o DHT20 e notifica o sistema de novas amostras.	5, para garantir leituras e notificações rápidas
task_display	Atualiza o ecrã TFT com os dados atuais, logs ou gráficos.	4, suficiente para atualizar o ecrã sem competir com o sensor

task_mqtt	Publica telemetria no broker MQTT e gere reconexões de rede.	Max priority – 3, garantindo fiabilidade nas comunicações com o broker mesmo quando o sistema está com muito tráfego
task_brightness	Ajusta o brilho do display com base na leitura analógica do potenciómetro.	4, para controlar o backlight de forma estável
task_button	Lê o botão físico com debouncing.	Idle priority + 1, pois faz apenas polling do botão
task_button_screen	Alterna entre os diferentes modos de visualização do ecrã.	Idle priority + 1, para evitar a interferência com tasks mais críticas
task_config_loader	Gere alterações nos ficheiros JSON e atualiza a configuração ativa.	5, para garantir que as configurações são carregadas sem serem bloqueadas
Led_task_blink	Responsável por controlar os padrões de piscar	1, suficiente para controlar o LED sem impactar o resto do sistema
Portal_delayed_reset_task	Espera antes de dar o reset forçado pelo utilizador, de maneira a reiniciar de forma segura	Idle priority + 3, assegurando que o reset é executado no timing pretendido

6.2.2 Comunicação entre tarefas

A comunicação entre tarefas é feita maioritariamente através de ***Task Notifications***, por serem leves e eficientes em sistemas FreeRTOS. Este mecanismo é usado para sinalizar a existência de novas leituras do sensor ao display e ao MQTT, bem como para notificar alterações de configuração. Desta forma, as tasks reagem apenas quando necessário, evitando uso excessivo de CPU e recursos.

6.2.3 Estruturas partilhadas na memória

As estruturas partilhadas são usadas para permitir a troca de informação entre várias tasks de forma segura e consistente. Estas estruturas centralizam dados críticos do sistema e evitam acessos concorrentes descontrolados. As estruturas são:

sensor_data_store
Guarda a última medição do sensor (temperatura, humidade e timestamp), protegida por mutex. Só a *task_sensor* escreve ou invalida os dados; o display, o servidor HTTPS e a task MQTT apenas leem para apresentar ou publicar a informação.

```
static sensor_data_t s_latest = {0};
static bool s_has_valid_data = false;
static SemaphoreHandle_t s_data_mutex = NULL;
static uint64_t s_last_timestamp_ms = 0;
```

Figura 6.2.3.1 — Estrutura partilhada das leituras dos sensores

system_status_snapshot_t

É o estado global do sistema (Wi-Fi, MQTT, SD, firmware, standby), protegido por mutex. Várias tasks e módulos atualizam partes do estado, enquanto o display, os servidores web e o core apenas consultam o snapshot para apresentar ou decidir comportamentos.

```
static system_status_snapshot_t s_state = {
    .wifi = {
        .sta_connected = false,
        .ip_addr = "desconhecido",
        .rssi = 0,
    },
    .mqtt = {0},
    .sd = {
        .mounted = false,
        .free_space_bytes = 0,
        .log_entries = 0,
        .last_log_ts = 0,
        .data_entries = 0,
        .data_last_log_ts = 0,
        .data_logging_enabled = false,
    },
    .general = {
        .uptime_seconds = 0,
        .firmware_version = "desconhecido",
        .device_uid = "desconhecido",
    },
    .standby_active = false,
};
```

Figura 6.2.3.2 — Estrutura partilhada do estado global do sistema

Configuração persistente (g_config_user + leitura_periodo_ms)

Mantém em RAM a configuração lida do config_user.json e o período ativo, com acesso sincronizado. Apenas o *config_manager* escreve; as restantes tasks usam estes dados para definir o período de leitura, Wi-Fi e comportamento geral.

```
config_user_t g_config_user = {
    .periodo_leitura_ms = CONFIG_DEFAULT_PERIOD_MS,
    .wifi_ssid = {0},
    .wifi_pass = {0},
    .mqtt_uri = {0},
    .mqtt_topic = "data",
    .mqtt_topic_events = "eventos",
    .mqtt_client_id = "esp32_default",
    .mqtt_username = {0},
    .mqtt_password = {0},
};
```

Figura 6.2.3.3 — Estrutura partilhada das configurações

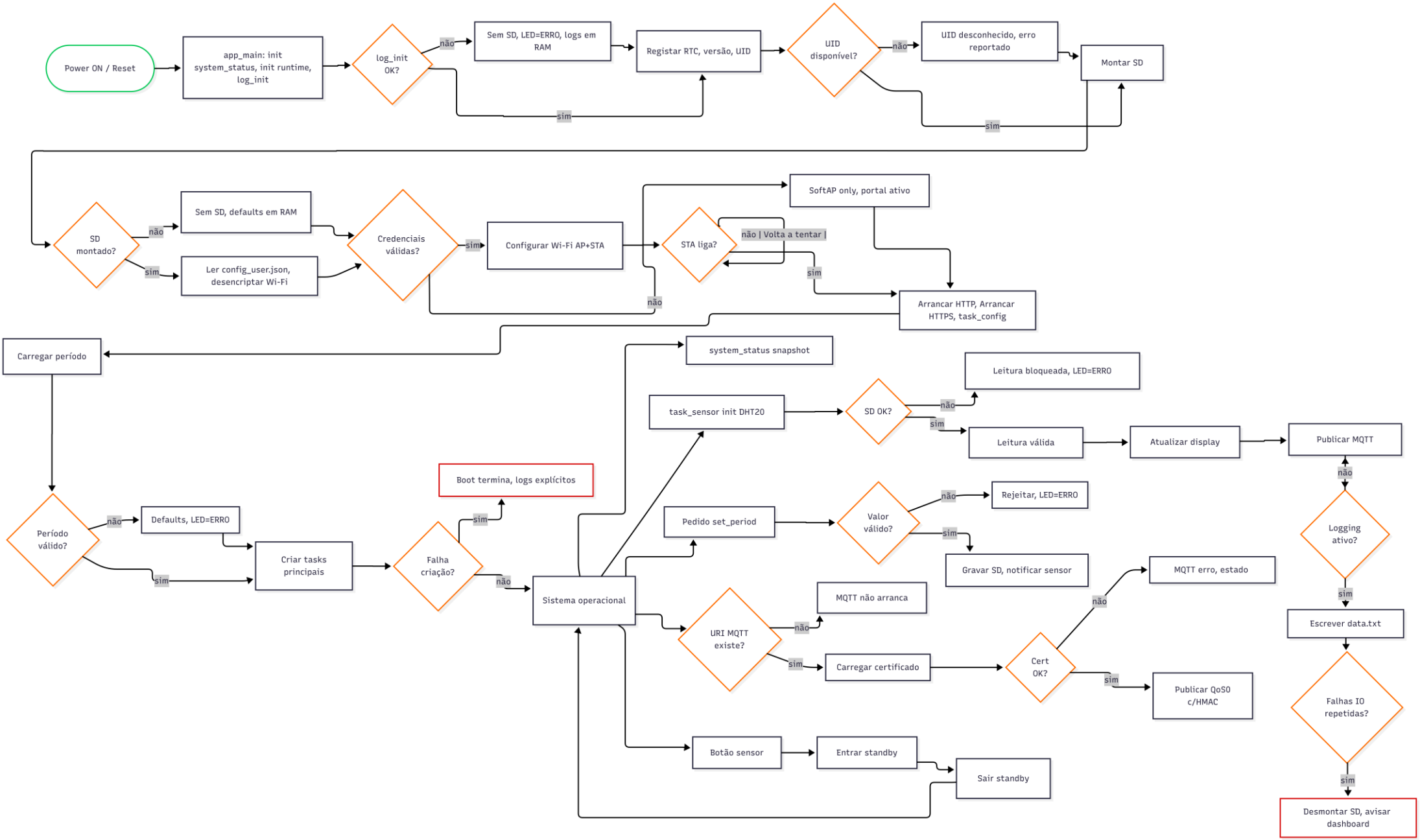
g_enable_sd_logging

Flag runtime que ativa/desativa logging no cartão SD, protegida por mutex. É lida por várias partes do sistema e só é alterada via endpoint HTTPS, refletindo-se imediatamente no estado global.

```
bool g_enable_sd_logging = true;                // Flag global protegida pelo mutex.
```

Figura 6.2.3.4 — Estrutura partilhada da flag dos logs

6.2.4 Fluxo global do sistema



7 Conclusão

O desenvolvimento deste projeto, aliado à realização deste relatório, permitiu-nos explorar o mundo emergente das soluções IoT. Aprofundámos o nosso conhecimento em temas já abordados em outras UC's, temas esses como a segurança informática, redes de comunicações, programação de microcontroladores, funcionamento de sensores/atuadores e sistemas eletrónicos. Além disso desenvolvemos novas capacidades, nomeadamente nas áreas das soluções autónomas e inteligentes, em sistemas operativos de tempo-real e gestão de tarefas.

Todo o conhecimento adquirido durante estas semanas será importante no nosso futuro, numa altura em que o desenvolvimento deste tipo de soluções se posiciona como uma componente fulcral do mercado de trabalho.